the end of the final block must be `last`, and it doesn't matter how many elements are in that block.

Once you've accumulated the results for the last block, you can wait for all the threads you spawned with `std::for_each` ❿, as in listing 2.7, and then add up the results with a final call to `std::accumulate` ⓫.

Before you leave this example, it's worth pointing out that where the addition operator for the type `T` is not associative (such as for `float` or `double`), the results of this `parallel_accumulate` may vary from those of `std::accumulate`, because of the grouping of the range into blocks. Also, the requirements on the iterators are slightly more stringent: they must be at least *forward iterators*, whereas `std::accumulate` can work with single-pass *input iterators*, and `T` must be *default constructible* so that you can create the `results` vector. These sorts of requirement changes are common with parallel algorithms; by their very nature they're different in some manner in order to make them parallel, and this has consequences on the results and requirements. Parallel algorithms are covered in more depth in chapter 8. It's also worth noting that because you can't return a value directly from a thread, you must pass in a reference to the relevant entry in the `results` vector. Alternative ways of returning results from threads are addressed through the use of *futures* in chapter 4.

In this case, all the information required by each thread was passed in when the thread was started, including the location in which to store the result of its calculation. This isn't always the case: sometimes it's necessary to be able to identify the threads in some way for part of the processing. You could pass in an identifying number, such as the value of `i` in listing 2.7, but if the function that needs the identifier is several levels deep in the call stack and could be called from any thread, it's inconvenient to have to do it that way. When we were designing the C++ Thread Library we foresaw this need, and so each thread has a unique identifier.

2.5. Identifying threads